Lab 2.2 - Build the stack (Developers)

Goal

Learn what goes where by building it: AGENTS.md, a path rule, a custom agent, a prompt file and an MCP server. Then prove each piece loads when it should, and only then.

You're done when

- ☐ The "where does it go?" table (step 1) is filled, with a one-line reason per row
- ☐ `AGENTS.md` is 40 lines or less, and you decided what (if anything) stays in `.github/copilot-instructions.md`
- ☐ You proved the legacy rule loads for a legacy file and does **not** load for a web file
- ☐ You saw what happens when two rule files disagree, and fixed it
- ☐ The `legacy-explorer` agent can read but cannot edit, and you proved it
- ☐ The agent read WI-1042 through a read-only MCP server

Option A - Guided

1. Where does it go? (5 min, no AI). For each rule, pick its home: **AGENTS.md**, **copilot-instructions.md**, **path rule** (`*.instructions.md`), **prompt file**, **skill**, **custom agent**, **MCP server**, **hook**, or **nowhere**. More than one answer can be right. Write one line on why.

#	Rule	Home	Why
a	Build with `dotnet build OrganLens.sln`, web tests with `dotnet test src/OrganLens.Api.Tests`		
b	Legacy projects are C# 7.3 / .NET 4.8: no records, no switch expressions		
c	Never log `PatientName` or any other patient identifier		
d	Never add a NuGet package without asking		
e	Our 8-point PR review checklist		
f	"Map one module: purpose, callers, tests, smells"		
g	Read work items from our board		
h	Exploring the code must never edit files		
i	Kidney volume uses π/6		
j	Use Copilot's Plan agent output format for plans		

2. AGENTS.md: generate, then cut (8 min). Agent mode:

```
/init
```

Then **Agent** mode:

```
Move the useful content of the generated instructions into AGENTS.md at the repo root.
Maximum 40 lines. Keep only: build and test commands, a 5-line map of the solution,
hard rules (no PHI in logs, no new packages without asking), where tests live.
Delete every line that only repeats what the code already shows.
For each line you keep, tell me the file that proves it is true.
```

Now decide: delete `.github/copilot-instructions.md`, or keep a few lines that only make sense for Copilot? Write down why. (Hint: other agents read `AGENTS.md`. Only Copilot reads `copilot-instructions.md`.)

3. A path rule (5 min). Create `.github/instructions/legacy.instructions.md`:

```
---
applyTo: "src/OrganLens.Legacy.*/**"
description: "Rules for the .NET Framework 4.8 legacy projects"
---
- C# 7.3 only: no records, no switch expressions, no `is not`, no target-typed `new()`.
- .NET Framework 4.8 APIs only. No new NuGet packages.
- Business logic goes in OrganLens.Legacy.Core, never in WinForms event handlers.
- Before changing a calculation, find every other copy of it (legacy and web) and list them.
```

4. Prove it loads, and only where it should (5 min). New chat, **Ask** mode, with `src/OrganLens.Legacy.WinForms/MainForm.cs` open:

```
Which C# language features may I use in this file? Which rule files did you use to answer?
```

Expand **References** at the top of the answer. Is `legacy.instructions.md` there? Now open `src/OrganLens.Api/Services/AnalysisService.cs`, start a new chat, ask the same question. The legacy rule must **not** appear. If it does, your `applyTo` glob is wrong.

5. Make two files disagree (5 min). Add this line to `AGENTS.md`:

```
- Test names: Should_<Result>_When_<Scenario>
```

And this line to `.github/copilot-instructions.md` (create it if you deleted it):

```
- Test names: <Method>_<Scenario>_<Result>
```

Ask 3 times, each in a new chat:

```
Suggest a name for a test that checks GET /api/studies/SYN-9999 returns 404.
```

Write down what you got. Both files were sent. The model picked one, and maybe not the same one each time. Now delete one of the two lines. Rule: **one home per rule.**

6. A read-only custom agent (7 min). Create `.github/agents/legacy-explorer.agent.md`:

```
---
name: legacy-explorer
description: Read-only explorer for the OrganLens legacy code. Maps and cites. Never edits.
tools: ['read', 'search']
---
You explore code. You never edit files and never run commands that change anything.
Every claim cites file + method. Write UNKNOWN when you cannot confirm something.
End every answer with: "Spot-check at least two of my citations."
```

Pick **legacy-explorer** in the agents dropdown. Ask:

```
Add a comment at the top of MainForm.cs that says "legacy".
```

It should not be able to. Then point your prompt file from Lab 2.1 at it: add `agent: legacy-explorer` to the frontmatter of `.github/prompts/map-module.prompt.md`, and run `/map-module` with `OrganLens.Legacy.Data`.

> Tool names differ between versions. Type # in the chat box to see the exact names in yours.

7. A read-only MCP server (8 min). Create `.vscode/mcp.json`:

```
{
  "servers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/",
      "headers": { "X-MCP-Readonly": "true" }
    }
  }
}
```

Click **Start** above the server entry and sign in. Hover the **context-window control** in the chat box before and after: how many tokens did the tool definitions add? **Agent** mode:

```
Using the GitHub MCP server, read issue WI-1042 in CodeValue/training-philips.
Quote it exactly. Then compare it with docs/briefs/WI-1042.md and list what the issue is
missing.
Do not modify the issue.
```

Ask it to add a comment to the issue. The call should fail: read-only.

> Azure DevOps instead? A stdio server: `"command": "npx", "args": ["-y", "@azure-devops/mcp", "<your-org>"]`.

Option B - Challenge

Build the same stack without the snippets above, plus:

- A second path rule for test code. Prove that it loads for `StudiesEndpointTests.cs` and not for `MeasurementService.cs`.
- Rule d ("no new packages") is only a request. Make it a law: write a hook in `.github/hooks/` that denies `dotnet add package`. Read the hooks docs first. Which surfaces run it today?
- Try a nested `AGENTS.md` in `src/OrganLens.Legacy.Core/` instead of the path rule. Which agents read it? What did you have to switch on in VS Code?

Stop and check

- Name one rule that belongs in AGENTS.md and **not** in copilot-instructions.md. Why?
- What is the difference between telling the agent "never edit" in a prompt and removing the edit tool?
- Your MCP server costs context on every request. Would you keep it switched on all day?

Before you leave

Commit the stack to `main`, so later labs can use it:

```
git add AGENTS.md .github .vscode docs
git commit -m "Agent stack: AGENTS.md, rules, explorer agent, prompts, MCP"
```